RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Unit 02

Entity Relationship Modelling and Constraints in Database

Key Reference: **Click here to student friendly Reference**

DBMS Team

Outlines

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

1. ER model concepts, notation for ER diagrams,
2. Mapping constraints,reduction of ER diagrams to tables,
3. Entity integrity, referential integrity, Keys constraints, Domain constraints,

Till Here ..

[CIE 01 Examination [10 March 2025] - Syllabus Portion]

---

1. Relationship of higher degree Integrity constraints,
2. Relational algebra and relational calculus.

ER Model

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

An **Entity Relationship Diagram** is a diagram that represents relationships among entities in a database. It is commonly known as an ER Diagram

Entity: An Entity in a Database is a **real-world object** which can be described in terms of some features. For example, a box is a real-world object which can be described in terms of shape, size, and color.

Attributes: The attributes are the **characteristics** that describe the entity of the database. For example, shape, size, and color in the above case.

Relationship: Relationship is the **logical binding between the different entities** which exist in the Database. Suppose, two entities are Parent and Child. Then, the relationship would be 'gave birth to.' It means the parent 'gave birth to the child, so the parent and child are related in this way.

Reference: https://blog.knoldus.com/introduction-er-diagram/

Entities

- *Entity* - a thing (animate or inanimate) of independent physical or conceptual existence and *distinguishable*.
  In the University database context, an individual *student*, *faculty member*, a *class room*, a *course* are entities.

- *Entity Set* or *Entity Type*-
  Collection of entities all having the same properties.
  Student entity set – collection of all *student* entities.
  Course entity set – collection of all *course* entities.

Attributes

Each entity is described by a set of attributes/properties that have associated values

student entity

- *StudName* – name of the student.
- *RollNumber* – the roll number of the student.
- *Sex* – the gender of the student etc.

All entities in an Entity set/type have the same set of attributes.

Chosen set of attributes – amount of detail in modeling.

Attributes

Types of Attributes (1/2)

- Simple Attributes
  - having atomic or indivisible values.

    example: *Dept* – a string

    PhoneNumber – a ten digit number
- Composite Attributes
  - having several components in the value.

    example: *Qualification* with components

    (*DegreeName, Year, UniversityName*)
- Derived Attributes
  - Attribute value is dependent on some other attribute.

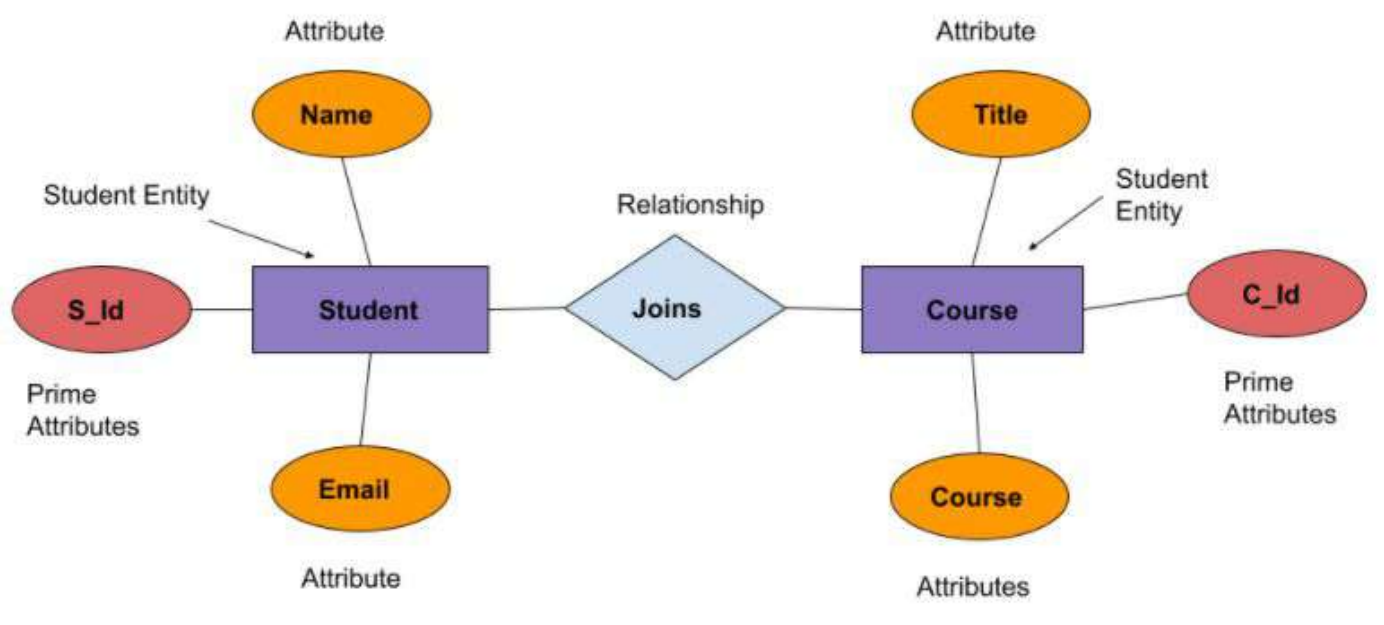
    example: *Age* depends on *DateOfBirth*.

    So age is a derived attribute.

Types of Attributes (2/2)

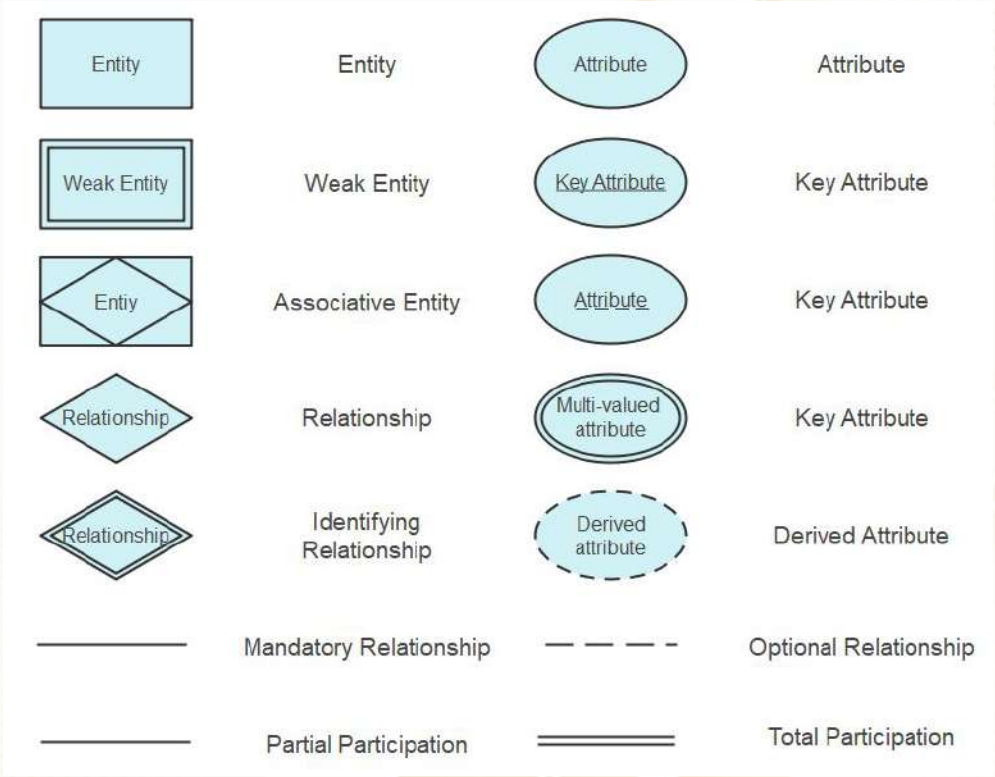
- Single-valued
  - having only one value rather than a set of values.
  - for instance, *PlaceOfBirth* – single string value.
- Multi-valued
  - having a set of values rather than a single value.
  - for instance, *CoursesEnrolled* attribute for student
    EmailAddress attribute for student
    PreviousDegree attribute for student.
- Attributes can be:
  - simple single-valued, simple multi-valued,
  - composite single-valued or composite multi-valued.

ER Model



Reference: https://blog.knoldus.com/introduction-er-diagram/

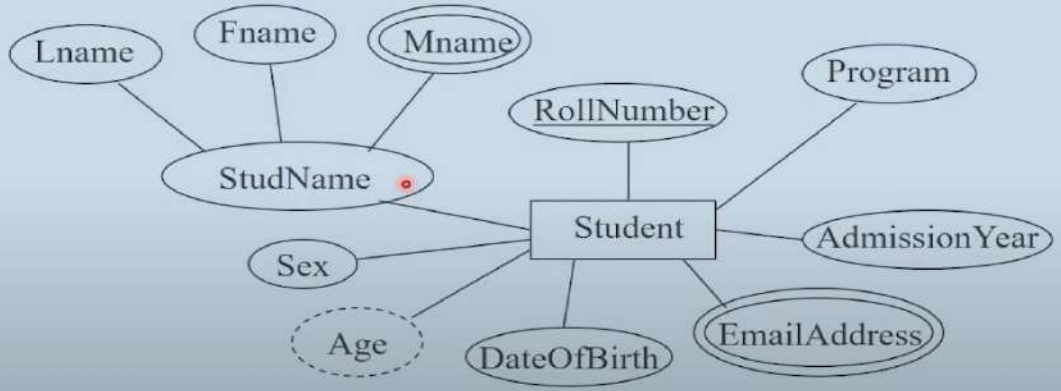
Notion of ER Diagrams



Notation for Entities

Diagrammatic Notation for Entities

entity - rectangle
attribute - ellipse connected to rectangle
multi-valued attribute - double ellipse
composite attribute - ellipse connected to ellipse
derived attribute - dashed ellipse



Domains of Attributes

Each attribute takes values from a set called its *domain*

For instance, *studentAge* – {17,18, …, 55}

HomeAddress – character strings of length 35

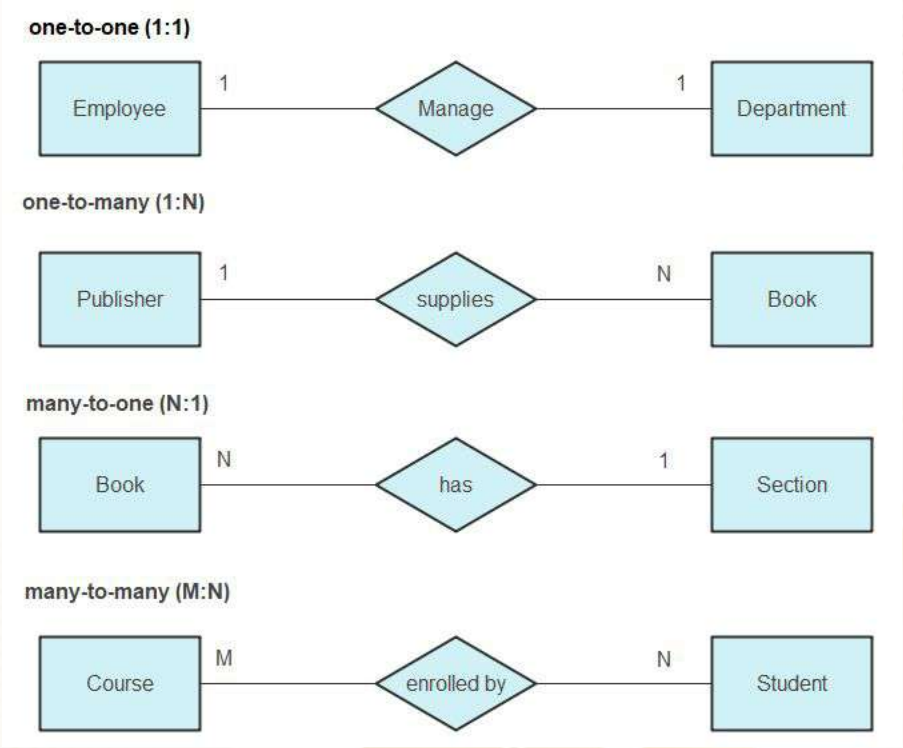
Domain of composite attributes –

cross product of domains of component attributes

Domain of multi-valued attributes –

set of subsets of values from the basic domain

Notion of ER Diagrams



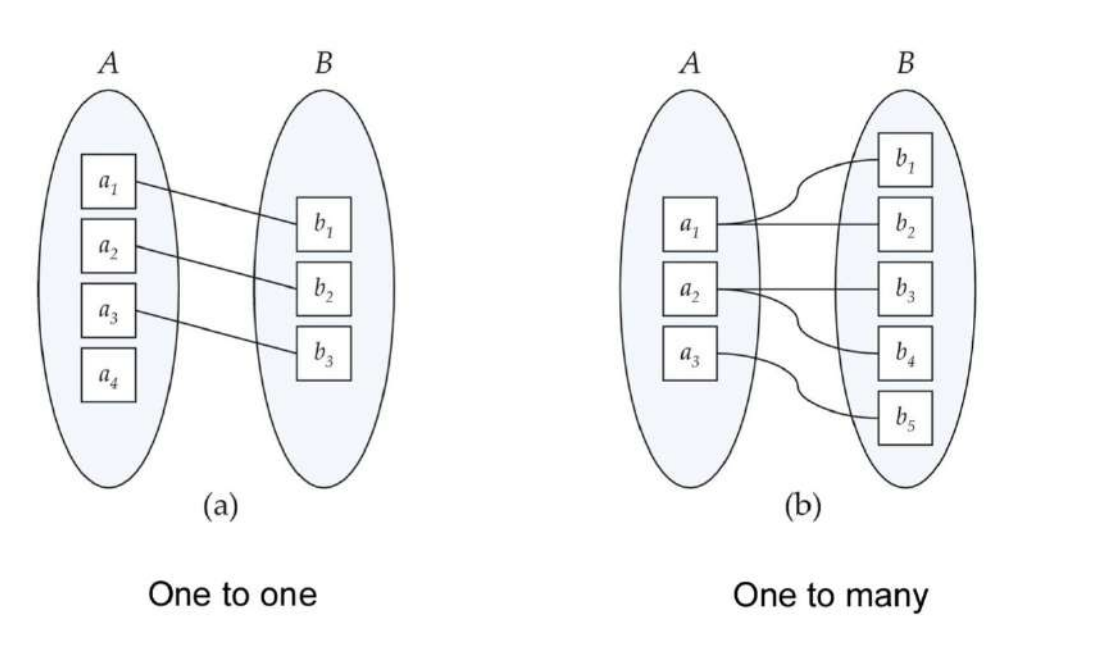
Mapping Cardinality Constraints

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Express the number of entities to which another entity can be associated via a relationship set.

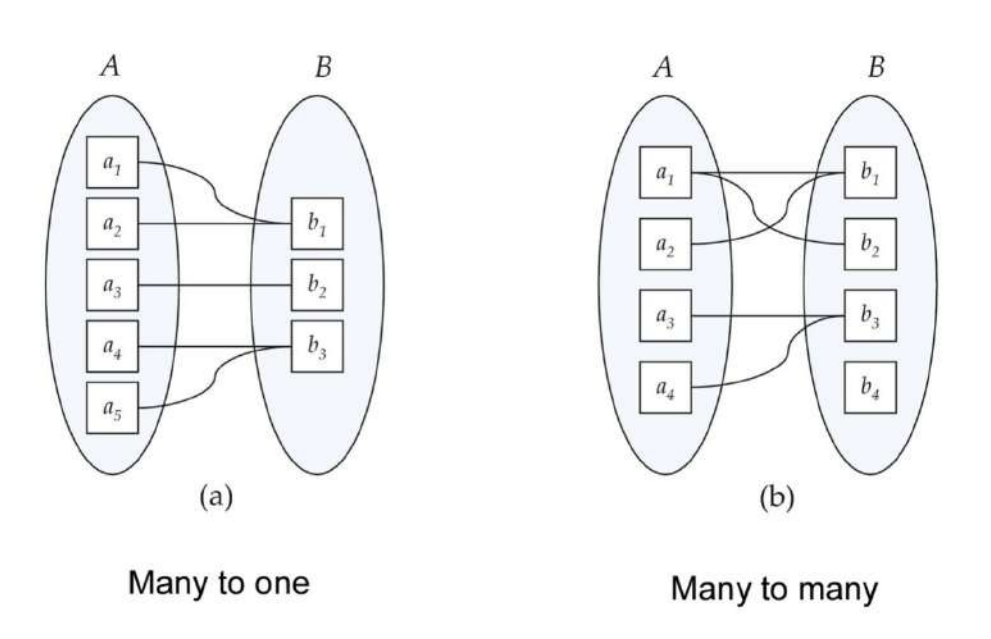
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
  - One to one
  - One to many
  - Many to one
  - Many to many

Mapping Cardinality



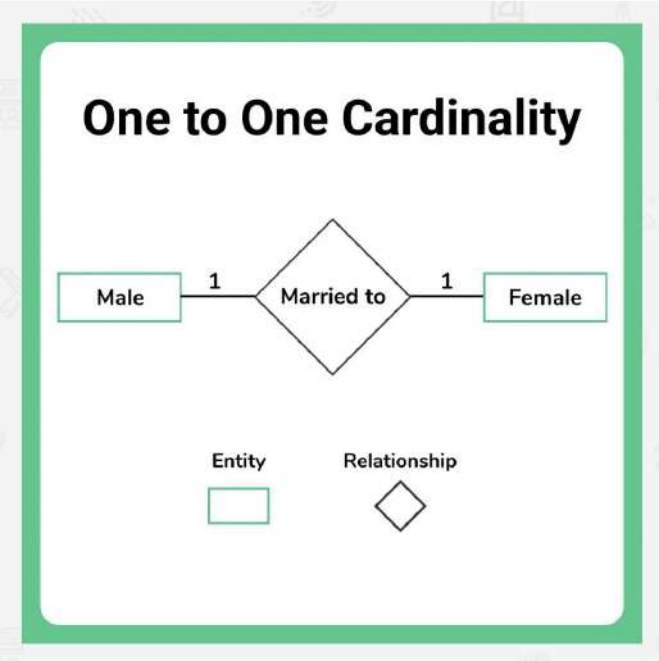
Note: Some elements in A and B may not be mapped to any elements in the other set

Mapping Cardinality



Note: Some elements in A and B may not be mapped to any elements in the other set

Mapping Cardinality

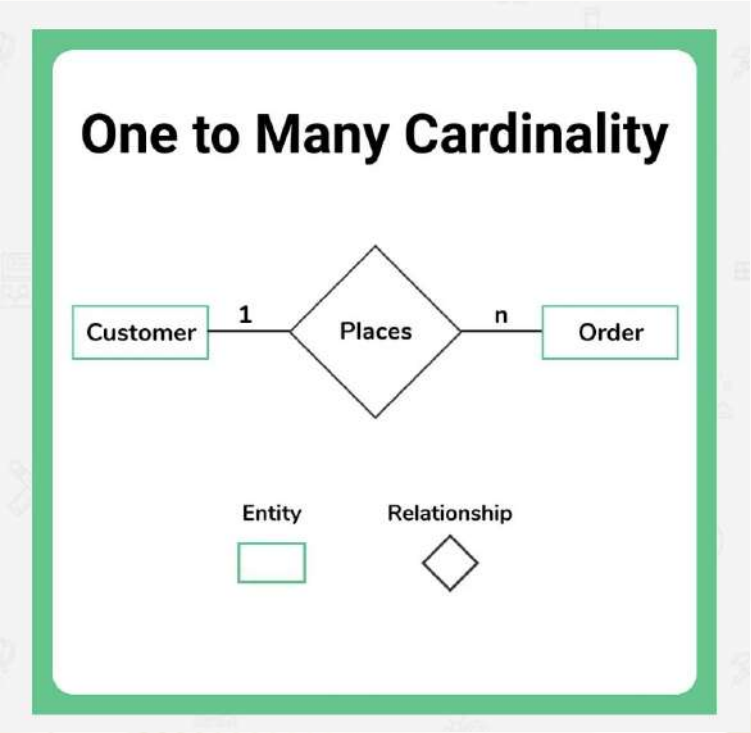


When a **single instance of an entity is associated with a single instance of another entity**, then it is called as one to one cardinality

Here each entity of the entity set participate only once in the relationship

Reference - https://prepinsta.com/dbms/mapping-constraints/

Mapping Cardinality

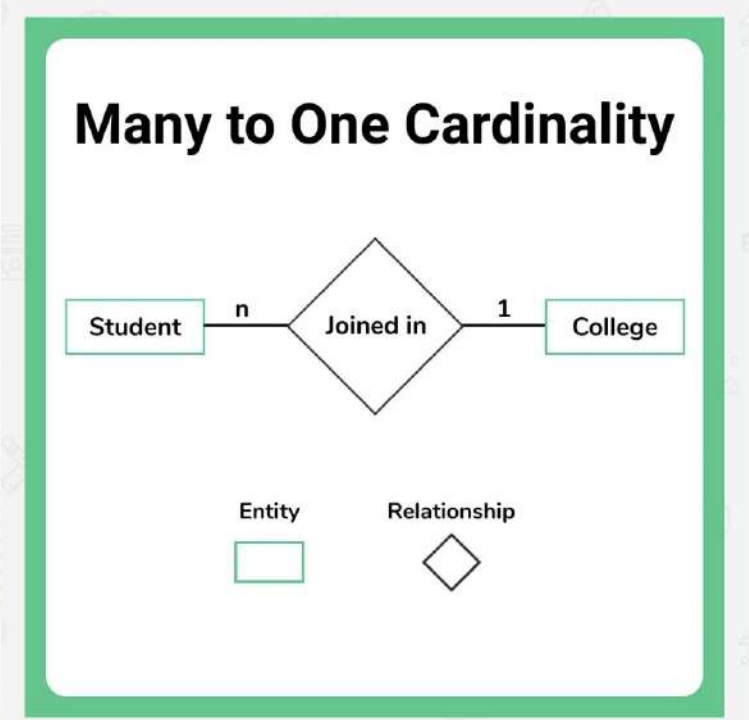


When is a **single instance of an entity is associated with more than one instance of another entity** then this type of relationship is called one to many relationships.

Here entities in one entity set can take participation in any number of times in relationships set and entities in another entity set can take participation only once in a relationship set.

Reference - https://prepinsta.com/dbms/mapping-constraints/

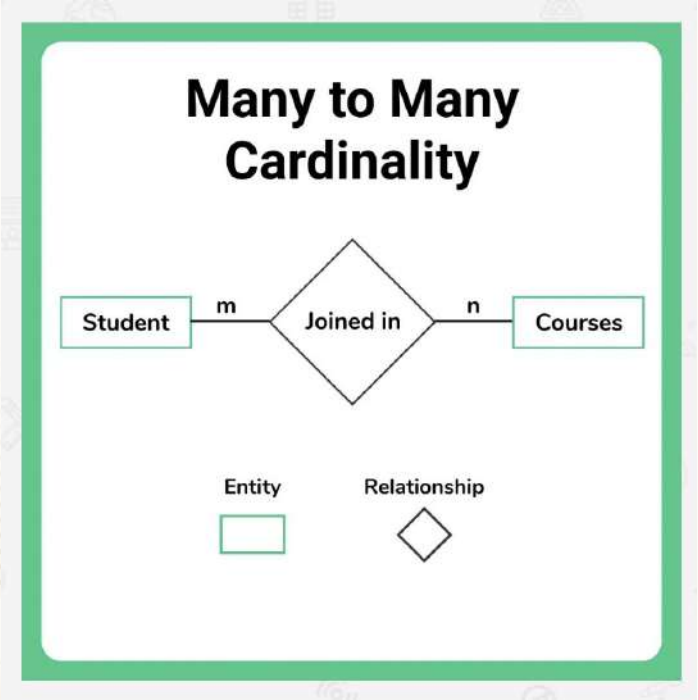
Mapping Cardinality



When entities in **one entity set can participate only once in a relationship set and entities in another entity can participate more than once in the relationship set**, then such type of cardinality is called many-to-one

Reference - https://prepinsta.com/dbms/mapping-constraints/

Mapping Cardinality

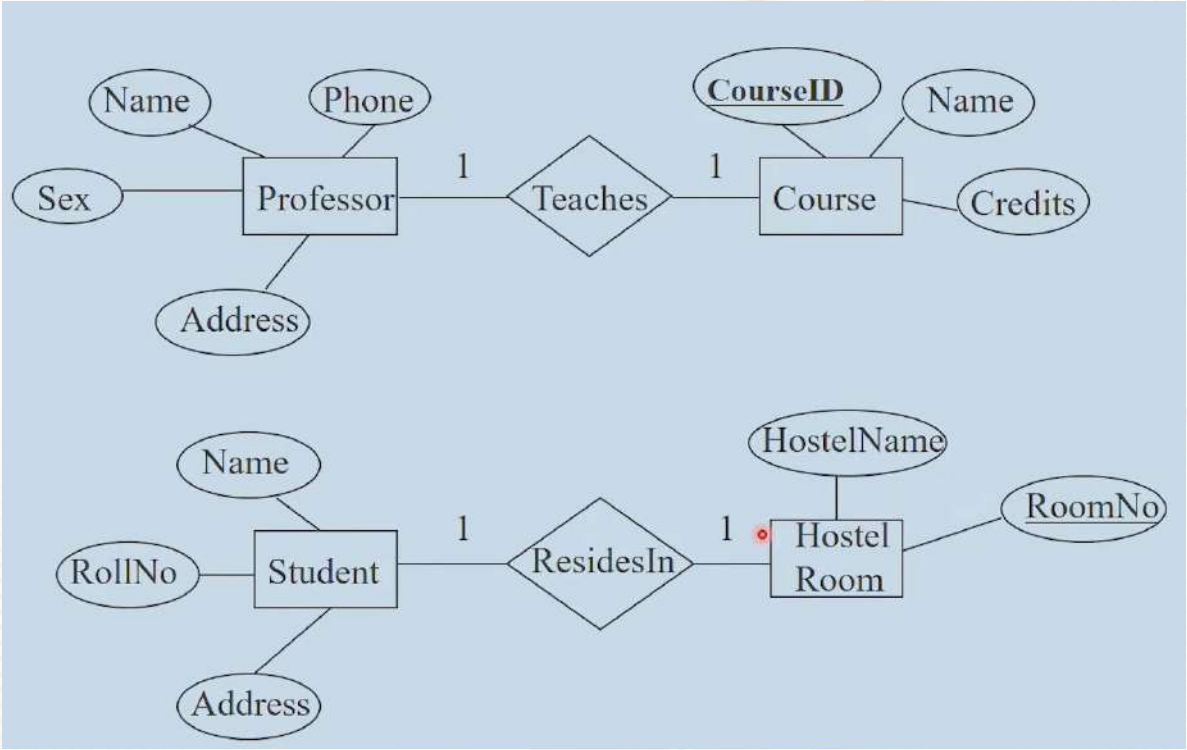


Here, **more than one instance of an entity is associated with more than one instance of another entity** then it is called many to many relationships.

In this cardinality, entities in all entity sets can take participate any number of times in the relationship cardinality is many to many.

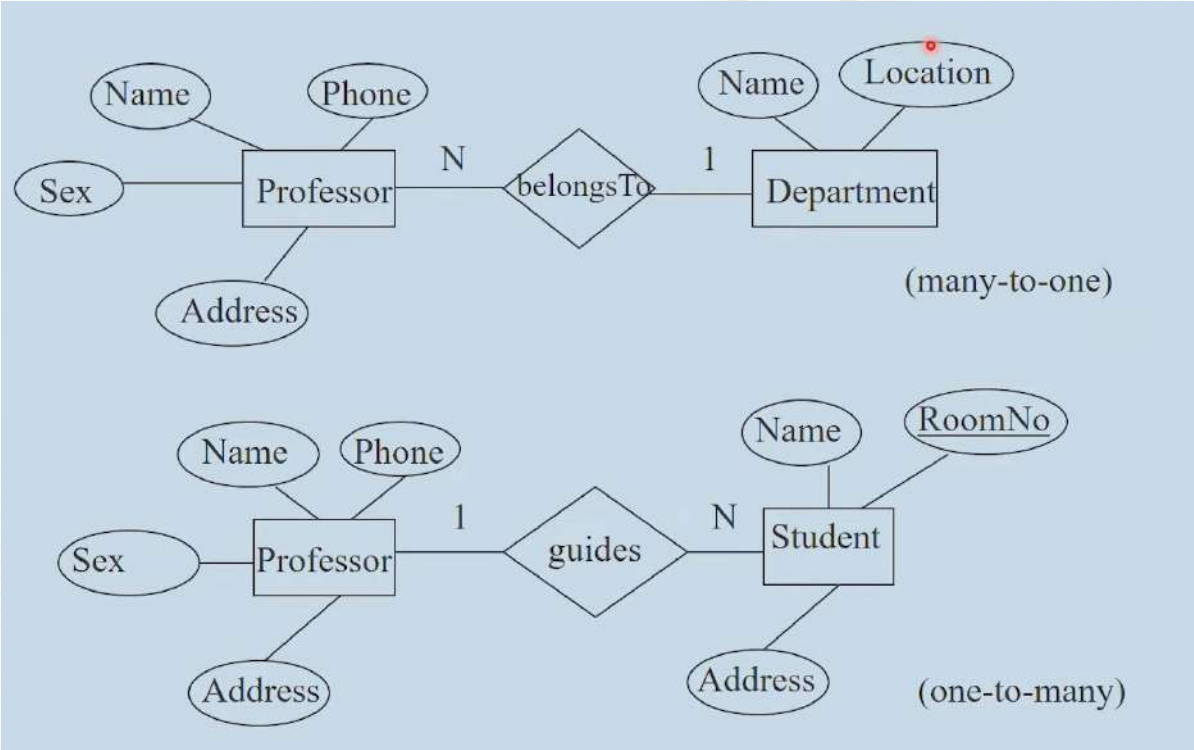
Reference - https://prepinsta.com/dbms/mapping-constraints/

ER Model – Example



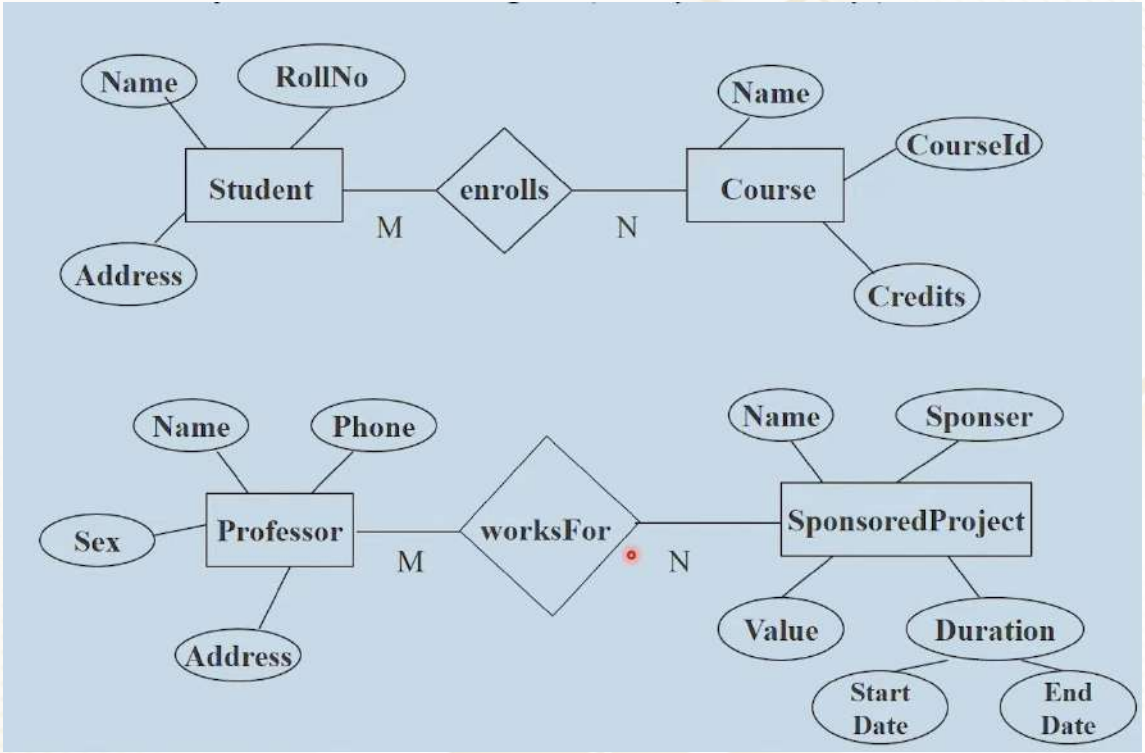
Reference - https://www.youtube.com/@nptel-nociitm

ER Model – Examples



Reference - https://www.youtube.com/@nptel-nociitm

ER Model


Name
RollNo
Student
Address
enrolls
M
N
Name
CourseId
Course
Credits
Name
Phone
Sex
Professor
Address
M
worksFor
N
Name
Sponser
SponsoredProject
Value
Duration
Start Date
End Date

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Click here for practice questions

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Additional References:

1. Database Design Phase - Maria DB [Click the link for Details]

1. https://huridocs.org/community-resources/designing-your-conceptual-data-model/ [Click the link for Details] - **Important**

1. https://www3.gmu.edu/schools/vse/seor/studentprojects/graduate/2014Fall/Cornerstones/Design.html [Click the link for Details]

Integrity Constraints Motivation

An integrity constraint is a condition specified on a database schema that restricts the data that can be stored in an instance of the database.

WHY AND HOW ?

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Why: Integrity Constraints **[ICs]** help prevent entry of incorrect information

How: DBMS enforces integrity constraints
– Allows **only legal database instances** (i.e., those that satisfy all constraints) to exist
– Ensures that all necessary checks are always performed and avoids duplicating the verification logic in each application

Constraints in ER diagrams

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

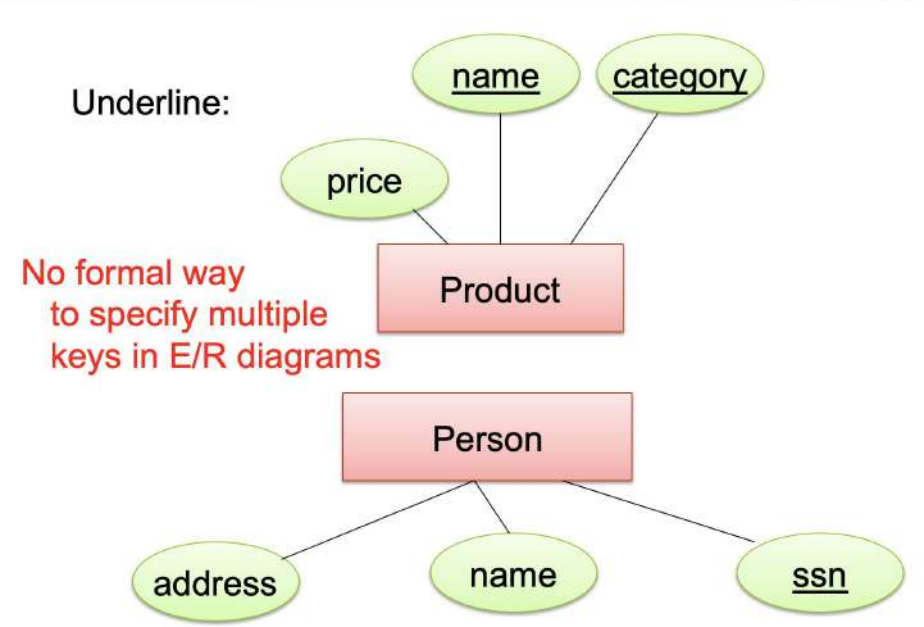
Finding constraints is part of the DBMS modeling process.

Commonly used constraints:

- **Keys:** social security number uniquely identifies a person.
- **Single-value constraints:** a person can have only one father.
- **Referential integrity constraints:** if you work for a company, it must exist in the database.
- **Other constraints:** peoples' ages are between 0 and 150.

References - https://courses.cs.washington.edu/courses/cse344/12wi/lectures/lecture15-constraints.pdf

Keys in ER Diagrams



References - https://courses.cs.washington.edu/courses/cse344/12wi/lectures/lecture15-constraints.pdf

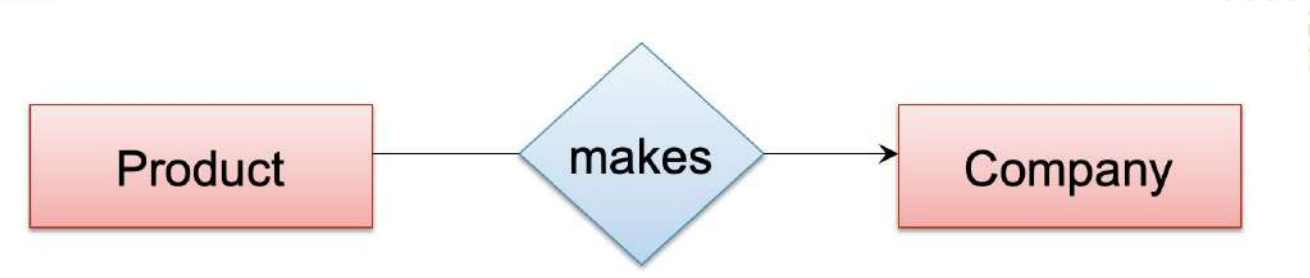
Single Values Constraints

v. s.

References - https://courses.cs.washington.edu/courses/cse344/12wi/lectures/lecture15-constraints.pdf

Referential Integrity Constraints

Each product made by **at most** one company. Some products made by no company



Referential Integrity Constraints

Each product made by **exactly** one company

Other Constraints

Types of Constraints in SQL

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

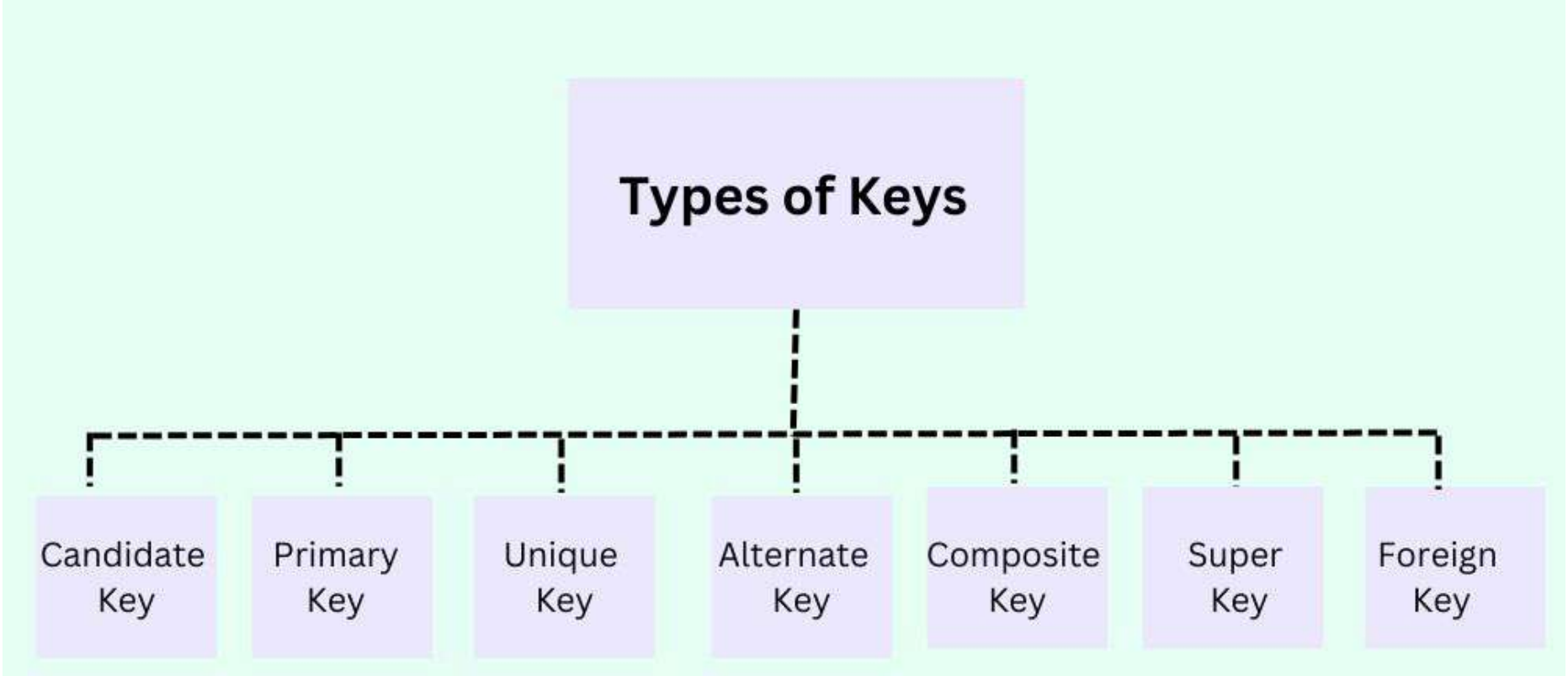
Focus

Constraints in SQL:

- Keys, foreign keys — simplest
- Attribute-level constraints
- Tuple-level constraints
- Global constraints: assertions — Most complex

- The more complex the constraint, the harder it is to check and to enforce

References - https://courses.cs.washington.edu/courses/cse344/12wi/lectures/lecture15-constraints.pdf


Types of Keys
Candidate Key
Primary Key
Unique Key
Alternate Key
Composite Key
Super Key
Foreign Key

Key Constraints

Product(name, category)

```
CREATE TABLE Product (
        name CHAR(30) PRIMARY KEY,
        category VARCHAR(20))
```

OR:

```
CREATE TABLE Product (
        name CHAR(30),
        category VARCHAR(20)
PRIMARY KEY (name))
```

Keys with multiple attributes

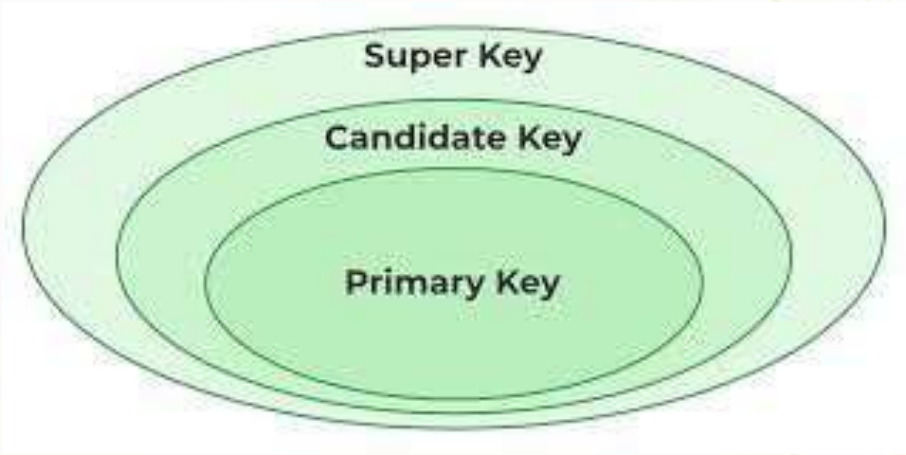
Product(name, category, price)

```
CREATE TABLE Product (
        name CHAR(30),
        category VARCHAR(20),
        price INT,
    PRIMARY KEY (name, category))
```

Name	Category	Price
Gizmo	Gadget	10
Camera	Photo	20
Gizmo	Photo	30
~~Gizmo~~	~~Gadget~~	~~40~~

Reference: https://db-book.com/slides-dir/PDF-dir/ch.pdf

Keys Overview



Reference: https://www.geeksforgeeks.org/types-of-keys-in-relational-model-candidate-super-primary-alternate-and-foreign/

Candidate Key

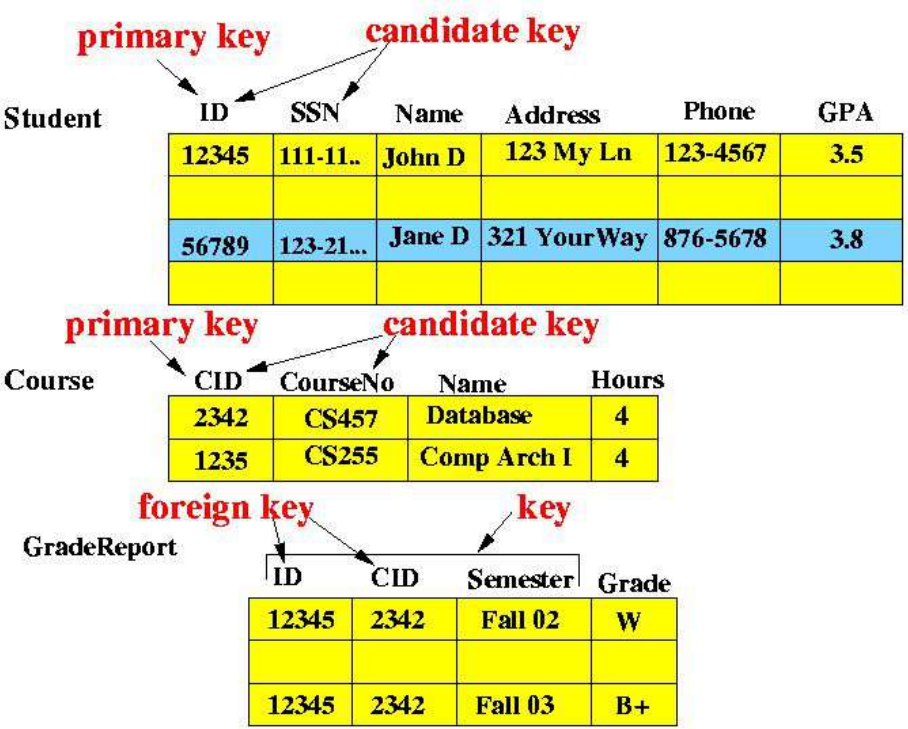
RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

A **candidate key** in a database management system (DBMS) is a **unique identifier for a record within a table** that can be **chosen as the primary key**.

It possesses the essential characteristics required for a primary key: uniqueness and minimal redundancy.

Reference: https://db-book.com/slides-dir/PDF-dir/ch.pdf

Candidate Key



Reference: https://db-book.com/slides-dir/PDF-dir/ch.pdf

Super Key

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

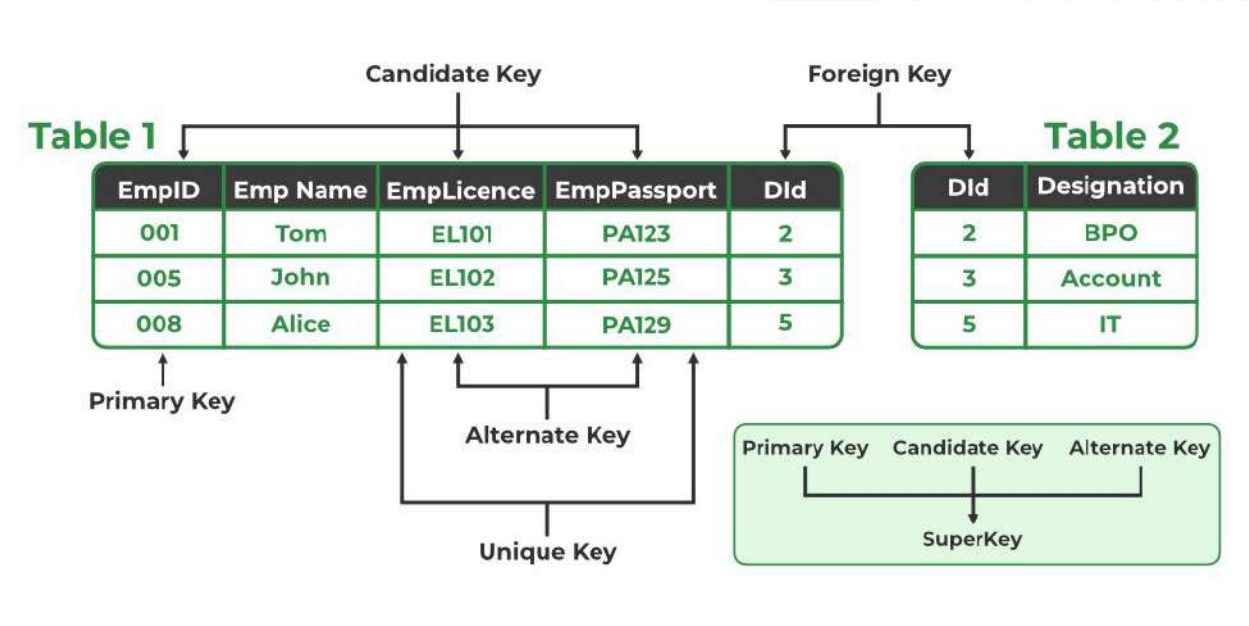
Super keys are collections of **one or more properties (columns)** in database management systems that allow a tuple (row) in a relation (table) to be distinctly identified

Role of Super Key :

- **Originality**: A super key ensures each value combination is unique, enabling accurate data retrieval and processing.
- **Selection**: Super keys help locate specific tuples in a dataset by utilizing their unique properties.
- **Security of Data**: They maintain data consistency by preventing duplicate and inconsistent entries.
- **Validity of Reference**: Super keys can serve as foreign keys, ensuring reliable references between related tables.
- **Indexing**: Super keys enhance database performance by enabling efficient indexing and faster data access.

Reference: https://db-book.com/slides-dir/PDF-dir/ch.pdf

Super Key



Reference: https://db-book.com/slides-dir/PDF-dir/ch.pdf

Super Key

order_id	customer_id	product_id	quantity
1	101	1	2
2	102	2	1
3	101	3	3
4	103	1	1
5	102	4	2

Identify the Super Key from the above table.

Primary Key

- A table's **primary key is a column that identifies each tuple (row)**.
- The table's primary key **enforces integrity restrictions**.
- A table may only contain one main key.
- **Duplicate and NULL values** cannot be used in the primary key.

Note: Choosing the primary key value in a table carefully is important because it can change quite infrequently

Primary Key

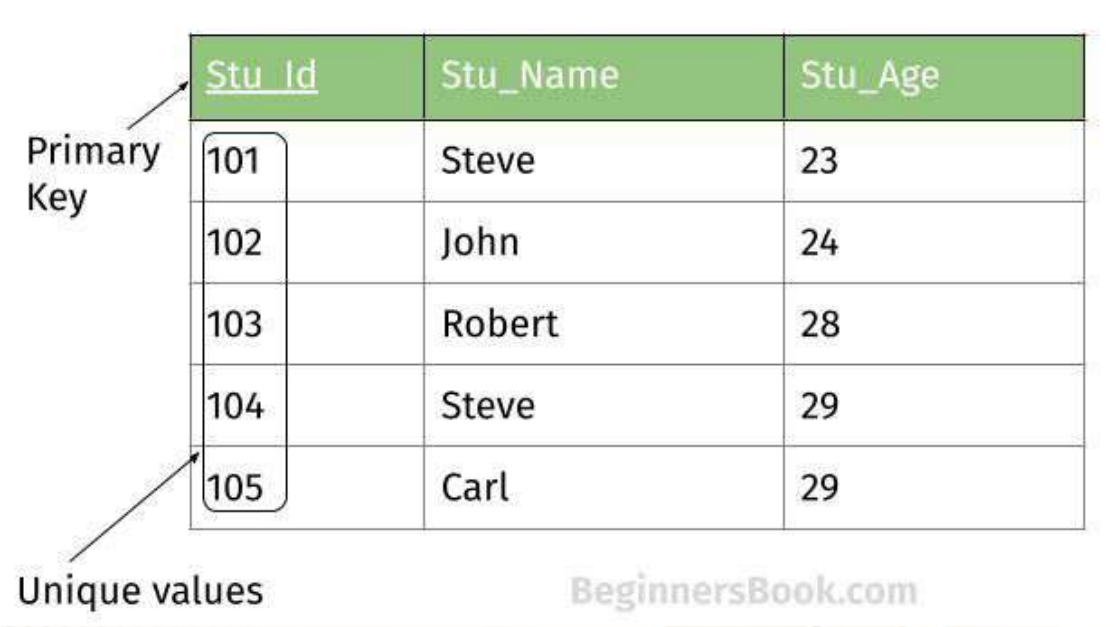
RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

The following are the main fundamental characteristics:

- Duplicate values are not permitted in the primary key column.
- It implements the entity integrity of the table.
- A table may only have one primary key column.
- From one or more table fields, we can create the primary key.
- **NOT NULL** constraints should be present in the primary key column.

Reference: https://db-book.com/slides-dir/PDF-dir/ch.pdf

Primary Key



Stu_Id	Stu_Name	Stu_Age
101	Steve	23
102	John	24
103	Robert	28
104	Steve	29
105	Carl	29

Unique Key

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Unique key constraints can **uniquely identify an individual tuple in a relation** or table. Unlike the main key, **a table may have several unique keys**.

Note: Only one NULL value per column is permitted for unique key restrictions

Unique Key

Candidate_Detail

Unique Key

Unique Key

Candidate_no	Name	Aadhar_no	Other_Id
2018101	X	1432113211621	4361
2018211	Y	Null	8121
2018121	Z	842191428136	Null

Unique Key

The following are the fundamental distinctive characteristics:

- One or more table fields can be used to create the unique key.
- There may be several distinct key columns defined in a table.
- Unclustered unique indexes are where a unique key is, by default, found.
- The column with the unique constraint may store a NULL value, but only one NULL is permitted per column.
- The unique constraint can be referred to as the foreign key to maintaining a table's uniqueness.

Primary Key	Unique Key
Used to serve as a unique identifier for each row in a table.	Uniquely determines a row which isn't primary key.
Creates clustered index	Creates non-clustered index
Cannot accept NULL values.	Can accepts NULL values.
A Primary key supports auto increment value.	A unique key does not supports auto increment value.
We cannot change or delete values stored in primary keys.	We can change unique key values.
Only one primary key	More than one unique key

Foreign Key Constraint

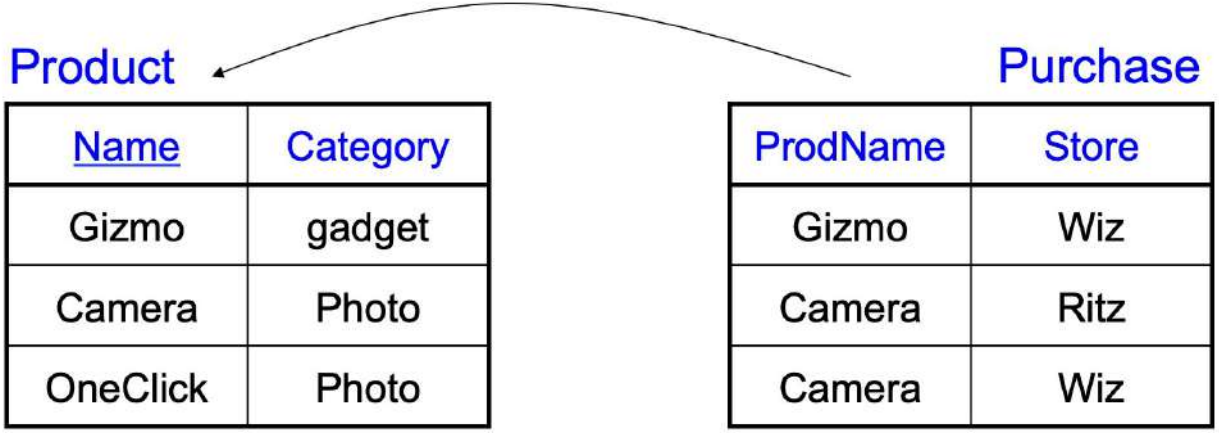
A **foreign key (FK)** is a column or combination of columns that is used to establish and enforce a link between the data in two tables to control the data that can be stored in the foreign key table

Foreign Key Constraint


```
CREATE TABLE Purchase (
prodName CHAR(30) REFERENCES Product(name),
date DATETIME
);
```

- **prodName** is a foreign key to Product(name)
- **name must be a key in Product**

Foreign Key Constraint

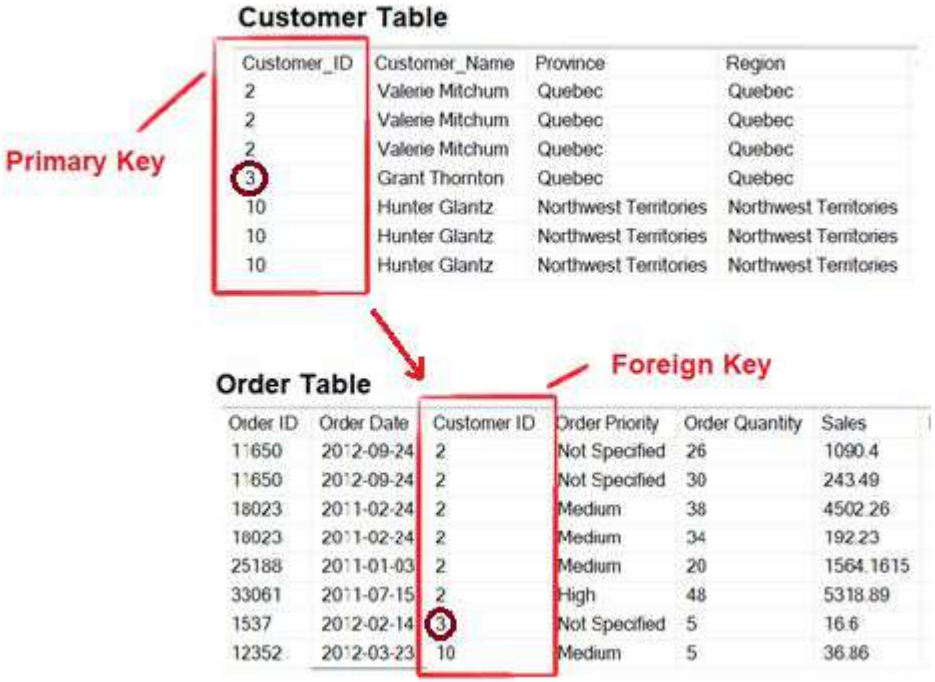


Name	Category
Gizmo	gadget
Camera	Photo
OneClick	Photo

ProdName	Store
Gizmo	Wiz
Camera	Ritz
Camera	Wiz

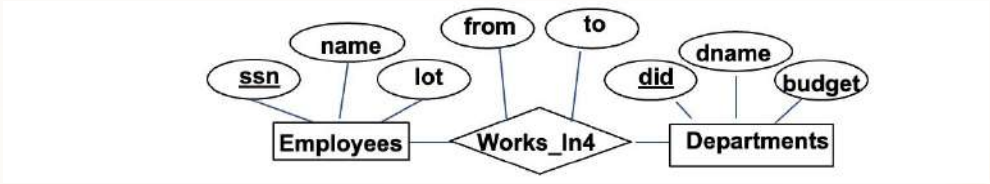
Reference: https://db-book.com/slides-dir/PDF-dir/ch.pdf

Foreign Key Constraint



Reference: https://db-book.com/slides-dir/PDF-dir/ch.pdf

- Consider the following ER diagram:

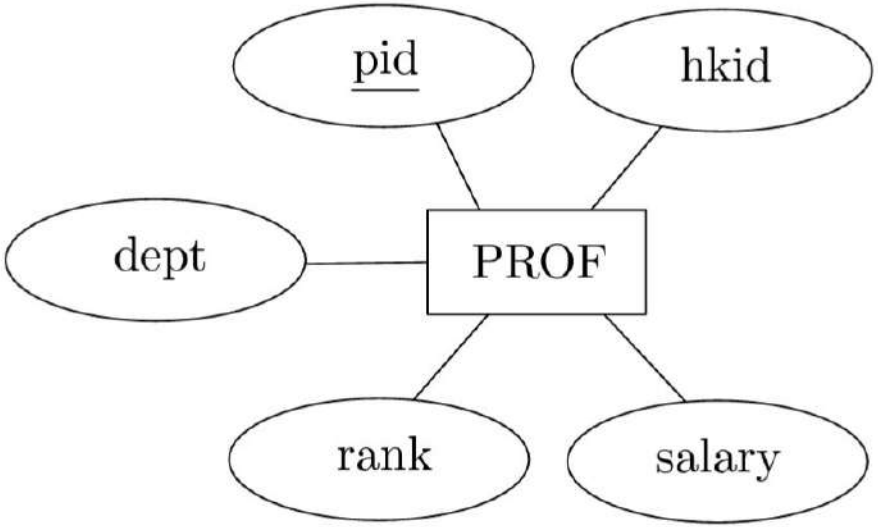


- A problem: Works_In4 does not allow an employee to work in a department for two or more periods

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Reduction of ER Diagrams to Table

Reduction of ER Diagrams to Table



Reduction of ER Diagrams to Table

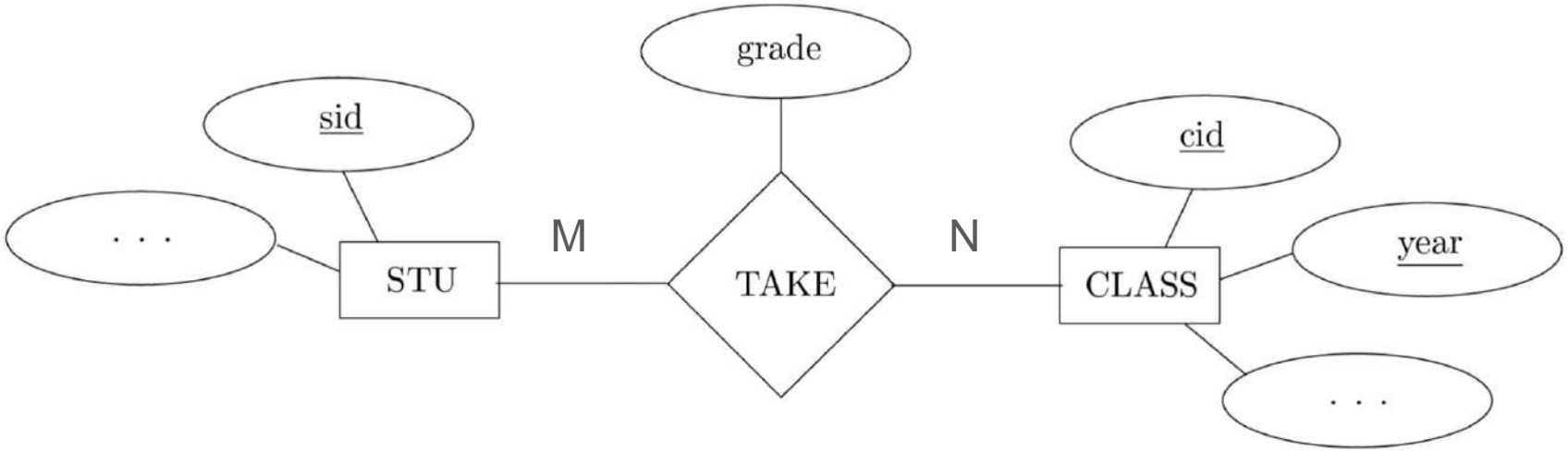
PROF (pid, hkid, dept, rank, salary),

and pid is set as a primary key

Reduction of ER Diagrams to Table

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Example:

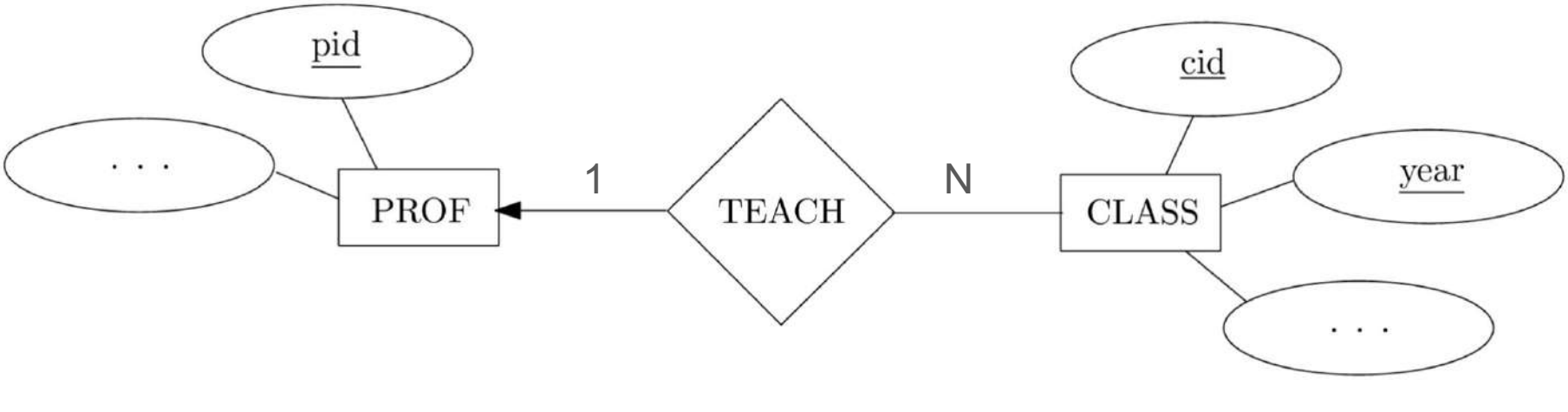


STUDENT(sid, ….)
CLASS(cid, ..)
TAKE (sid, cid, year, grade)
where the Foreign Key is (sid, cid).

Reduction of ER Diagrams to Table

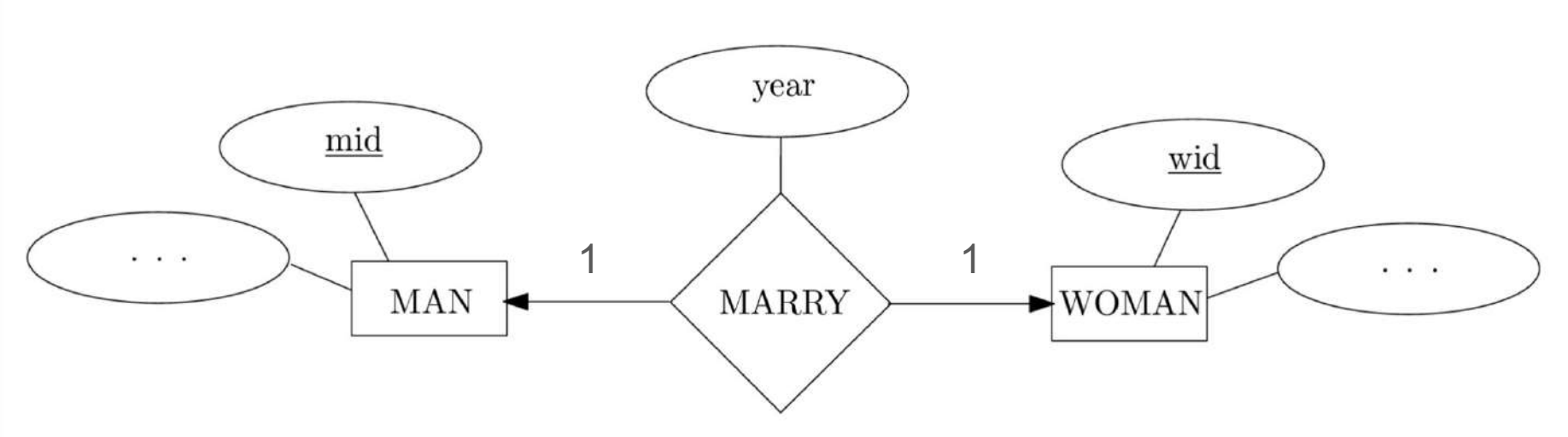
RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Example:

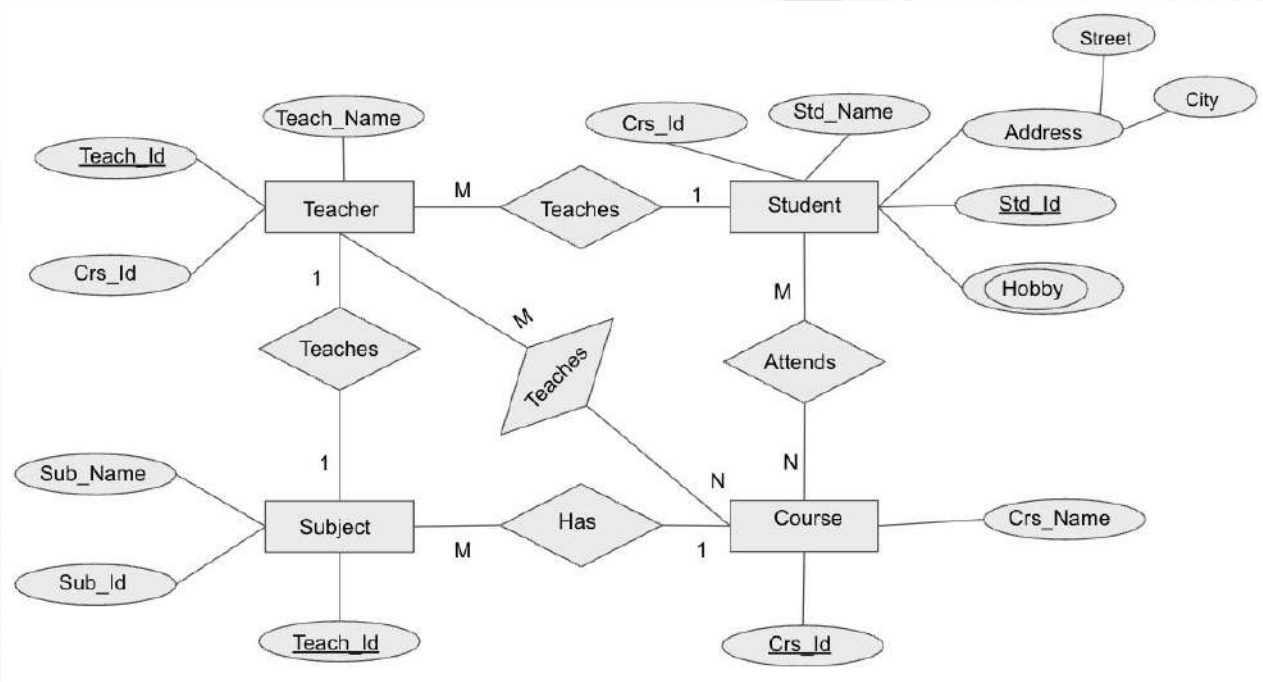


PROF(pid, ….)
CLASS(cid, year, pid ..)
where the Foreign Key is (pid).

Reduction of ER Diagrams to Table



Reduction of ER Diagrams to Table



Reduction of ER Diagrams to Table cont ..

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Tables

The tables are the **strong entities** that we have. Here we have the **STUDENT, TEACHER, SUBJECT, and COURSE**.

Reduction of ER Diagrams to Table cont ..

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Columns in each Table
The columns of a table are the attributes associated with that entity. Let's take a look at each table.

We will first consider the Simple attributes that are not Composite or Multivalued.

1. **Student Table:** Std_Id(KEY), Std_Name, DOB
2. **Teacher Table:** Teach_Id(KEY), Teach_Name
3. **Subject Table:** Sub_Id(KEY), Sub_Name
4. **Course Table:** Crs_Id(KEY), Crs_Name

Reduction of ER Diagrams to Table cont ..

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

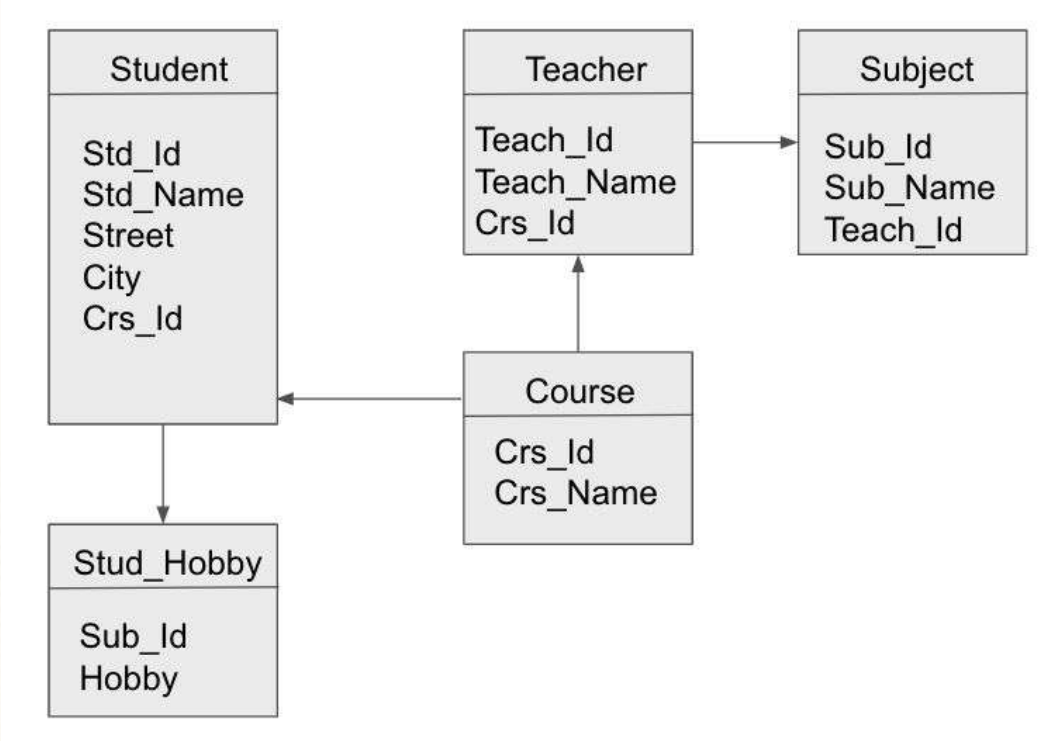
Now, **let's consider the Multivalued attributes like HOBBY.**

We will be having a **separate table for the Multivalued attribute** like **HOBBY**. Here we will consider the columns as Stud_Hobby and Std_Id. The Std_Id will associate the hobby with that particular student.

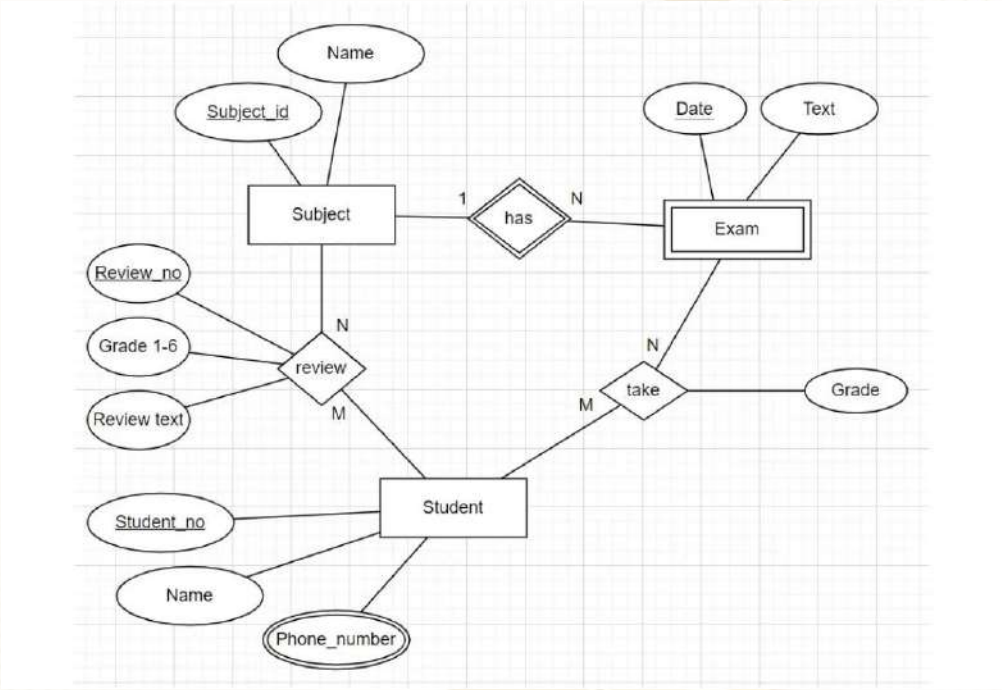
Composite Attributes

For composite attributes, like ADDRESS, we will have each of the CITY and STREET as a column in the Student Table.

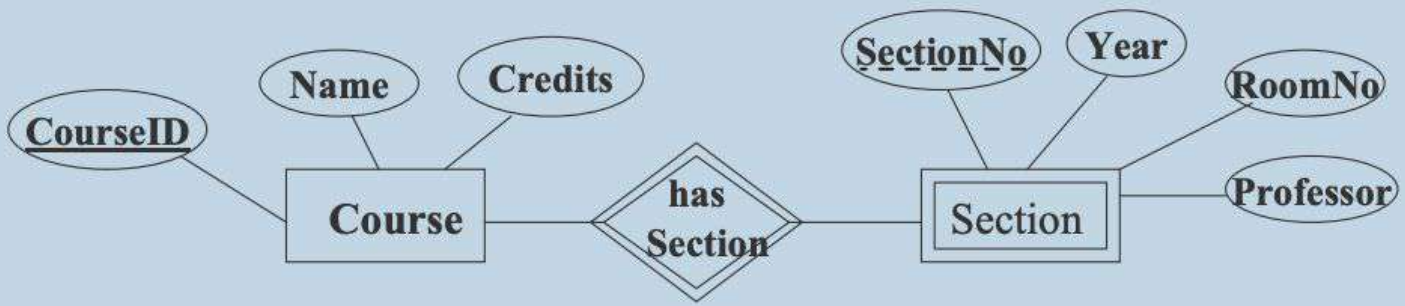
Final Table


Student
Std_Id
Std_Name
Street
City
Crs_Id
Teacher
Teach_Id
Teach_Name
Crs_Id
Subject
Sub_Id
Sub_Name
Teach_Id
Course
Crs_Id
Crs_Name
Stud_Hobby
Sub_Id
Hobby

Home Work Activity [Convert the ER Model into Relational Schema]



Additional example [Scenario of weak entity]



Corresponding tables are

course

courseId	name	credits

section

sectionNo	courseId	year	roomNo	professor

Primary key of *section* = {courseId, sectionNo}

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Practice Questions [ER Modelling] Link:
[20 Question + 35 MCQ]
https://docs.google.com/document/d/1Q1_JFvFRYkInLma9yWktLEUedpZlOHHTmlcJ6CQG_NE/edit?usp=sharing

Best reference video for very basic understanding ER Modelling :

Part 01 - https://youtu.be/xsg9BDiwiJE?t=1

Part 02 - https://youtu.be/hktyW5Lp0Vo?t=550

Portion Till CIE 01 —-------- All the Best !!

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Relational Algebra and Relational Calculus

Overview

Relational Algebra came in **1970** and was given by **Edgar F. Codd** (Father of DBMS).

It is also known as **Procedural Query Language(PQL)** as in PQL, a programmer/user has to mention two things,

"What to Do" and "How to Do".

Example

Suppose our data is stored in a database, then **relational algebra is used to access the data from the database**.

The **First thing** is we have to **access the data**, this needs to be specified in the query as "**What to Do**",

But we have to also **specify the method/procedure in the query** that is "**How to Do**" or how to access the data from the database.

Codd's Relational Algebra (RA)

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Data is stored as a set of relations

- Relations implemented as tables
- Tuple in a relation is a row in the table
- Attribute (from domain) in relation is column in table

RA = A set of mathematical operators that compose, modify, and combine tuples within different relations. Relations are sets.

- Mapping: maps elements in one set to another

Relational algebra operations operate on relations and produce relations (“closure”)

f: Relation à Relation

f: Relation x Relation à Relation

Preliminaries

A query is applied to **relation instances**, and the **result of a query is also a relation instance**.

- **Schemas of input** relations for a query are **fixed** (but query will run regardless of instance!)
- The **schema for the result** of a given query is also **fixed.**

Determined by definition of query language constructs.

Notation: Positional (R[0]) vs. named-field notation (R.name):

Positional notation easier for formal definitions, named-field notation more readable.

We will use named field notation R.name

- Both used in SQL

Relational Algebra

- Query language used to update and retrieve data that is stored in a data model.
- Relational algebra is a set of relational operations for retrieving data.

Every relational operator takes as input one or more relations and produces a relation as output.

- Closure property - input is relations, output is relations
- Unary operations - operate on one relation
- Binary operations - have two relations as input

A sequence of relational algebra operators is called a relational algebra expression.

Relational Algebra Operators

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Basic operations:

- **Selection** - Selects a subset of rows from relation.
- **Projection** - Deletes unwanted columns from relation.
- **Cross-product** - Allows us to combine two relations.
- **Set-difference** - Tuples in relation 1, but not in relation 2.
- **Union** - Tuples in relation. 1 or in relation. 2.

Note: cross-product, set-difference, union are the set operations.

Relational Algebra Expression: Syntax

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

RA operators operate on relations and produce relations –
closed algebra

- Defined recursively

Basic expression consists of a relation in the schema or a constant relation.

• What is a constant relation ?

(R) Let E1 and E2 be RA expressions, then

- (E1 ∪ E2) is a RA expression
- (E1 – E2) is a RA expression
- (E1 X E2) is a RA expression

σ_P (E1) is a RA expression

Where P is a predicate (conditional statement)

π_S(E1) is a RA expression

Where S is subset of the attributes in the schema

ρ_R (E1) is a RA expression

Operations Can be composed

- If R1, R2 are relations (sets), then R1 <op> R2 is also a relation (set).

Operations are defined as Set operations

- Input is a set, output is a set

SQL allows duplicated, RA does not

Operator Precedence

Just like mathematical operators, the relational operators have precedence.

The precedence of operators from highest to lowest is:

- unary operators - Selection, Projection , rename
- Cartesian product and joins - X,⋈ , division
- intersection
- union and set difference

Parentheses can be used to change the order of operations.

It is a good idea to always use parentheses around the argument for both unary and binary operators.

STUDENT

sid	name
1	Jill
2	Matt
3	Jack
4	Maury

Takes

sid	exp-grade	cid
1	A	550-0103
1	A	700-1003
3	A	700-1003
3	C	500-0103
4	C	500-0103

COURSE

cid	subj	sem
550-0103	DB	F03
700-1003	Math	S03
501-0103	Arch	F03

PROFESSOR

fid	name
1	Narahari
2	Youssef
8	Choi

Teaches

fid	cid
1	550-0103
2	700-1003
8	501-0103

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Projection Operator

Query the database and fetch only some column/attribute from the relation

Example: We want student name only from students table

$$\Pi_{name}(Student)$$

Projection Operator - Formal Definition

- Given a list of column names α and a relation R, π_α (R) extracts the columns in α from the relation.
- The projection operation on relation R with output attributes $A_1,\ldots,A_m$ is denoted by $\Pi_{A_1,\ldots,A_m}(R)$.

$$\Pi_{A_1,\ldots,A_m}(R)=\{t[A_1,\ldots, A_m] \mid t\in R\}$$

where

- R is a relation, t is a tuple variable
- $\{A_1,\ldots,A_m\}$ is a subset of the attributes of R over which the projection will be performed.
- Order of $A_1,\ldots, A_m$ is significant in the result.
- Cardinality of $\Pi_{A_1,\ldots,A_m}(R)$ is not necessarily the same as R because of duplicate removal.

Projection Example:

Emp Relation

eno	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

$\Pi_{eno,ename}(Emp)$

eno	ename
E1	J. Doe
E2	M. Smith
E3	A. Lee
E4	J. Miller
E5	B. Casey
E6	L. Chu
E7	R. Davis
E8	J. Jones

- Given a list of column names α and a relation R, π_α (R) extracts the columns in α from the relation. Output is relation…so removes duplicates!
- Example: find sid and grade from enrollment table

Takes

sid	exp-grade	cid
1	A	550-0103
1	A	700-1003
3	A	700-1003
3	C	500-0103
4	C	500-0103

$\Pi_{\text{sid,exp-grade}}$ (Takes)

sid	exp-grade
1	A
3	A
3	C
4	C

Note: duplicate elimination. In contrast, SQL returns by default a multiset and duplicates must be explicitly removed.

Practice Question (Projection Only)

1. Given a relation **Students**(StudentID, Name, Age, Grade), write a relational algebra expression to retrieve only the Name and Grade of all students.

1. Given a relation **Books**(BookID, Title, Author, YearPublished, Genre), write a relational algebra expression to retrieve the Title and Author of all books.

Selection Operation

We want to query table and fetch only the rows that satisfy come condition

Example: Students whose grade = B

Output relation has the same number of columns as the input relation, but may have less rows.

Select Operation σ

- Notation: $\sigma_{predicate}(Relation)$
- Relation: can be table or result of another query
- Predicate:
  1. Simple predicate
     - Attribute1 = attribute2
     - Attribute = constant (also >, <, >=, <=, <>)
  2. Complex predicate
     - Predicate AND predicate
     - Predicate OR predicate
     - NOT predicate
- Idea: select rows from a Relation based on a predicate

Complex Predicate Conditions

- Conditions are built up from boolean-valued operations on the field names.
  - exp-grade <> “A”, name = “Jill”, STUDENT.sid=Takes.sid
- RA allows comparison predicate on attributes
  - =, not=, >, <, >=, <=
- Larger predicates can be formed using logical connectives – *or* (∨) and *and* (∧) and *not* (¬)
- Selection predicate can include comparison between attributes

Selection Example

Emp Relation

eno	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

$\sigma_{title = 'EE'}(\text{Emp})$

eno	ename	title	salary
E1	J. Doe	EE	30000
E6	L. Chu	EE	30000

$\sigma_{salary > 35000 \vee title = 'PR'}(\text{Emp})$

eno	ename	title	salary
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

Logic operators: $\wedge$ AND, $\vee$ OR, $\neg$ NOT

Practice Question (Selection Only)

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

1. Given a relation **Employees**(EmpID, Name, Department, Salary), write a relational algebra expression to find all employees who work in the "Sales" department.

1. Given a relation **Customers**(CustomerID, Name, City, Country), write a relational algebra expression to find all customers who live in "Paris".

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Practice Questions [Selection and Projection - Both]

1. Given a relation Orders(OrderID, Product, Quantity, Price), write a relational algebra expression to retrieve the Product and Price of orders where the Quantity is greater than 10.

1. Given a relation Students(StudentID, Name, Age, GPA, Major), write a relational algebra expression to retrieve the Name and GPA of students who are majoring in "Computer Science" and have a GPA greater than 3.5

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Cross Product [X]

Operators that 'combine' relations

- Thus far, only operated on a single relation
- How to connect two relations ?
  - To find name of students taking a specific course with cid, we need to look at both students and Takes (enrolled) tables

- Operator(s) that produce a relation (set of tuples) after combining two different relations

- Set theory provides us with the cartesian product operator (between two sets; but can be applied to product of any number of sets – to get a k-tuple)

Product X

- "Join" is a generic term for a variety of operations that connect two relations. The basic operation is the cartesian product, R × S, *which concatenates every tuple in R with every tuple in S*. Example:

STUDENT

sid	name
1	Jill
2	Matt

SCHOOL

school
UPenn
GWU

STUDENT x SCHOOL

sid	name	school
1	Jill	UPenn
2	Matt	UPenn
1	Jill	GWU
2	Matt	GWU

Cartesian Product Example

Emp Relation

eno	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000

Proj Relation

pno	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	CAD/CAM	250000

Emp × *Proj*

eno	ename	title	salary	pno	pname	budget
E1	J. Doe	EE	30000	P1	Instruments	150000
E2	M. Smith	SA	50000	P1	Instruments	150000
E3	A. Lee	ME	40000	P1	Instruments	150000
E4	J. Miller	PR	20000	P1	Instruments	150000
E1	J. Doe	EE	30000	P2	DB Develop	135000
E2	M. Smith	SA	50000	P2	DB Develop	135000
E3	A. Lee	ME	40000	P2	DB Develop	135000
E4	J. Miller	PR	20000	P2	DB Develop	135000
E1	J. Doe	EE	30000	P3	CAD/CAM	250000
E2	M. Smith	SA	50000	P3	CAD/CAM	250000
E3	A. Lee	ME	40000	P3	CAD/CAM	250000
E4	J. Miller	PR	20000	P3	CAD/CAM	250000

Union ∪ (same as set union operation)

- If two relations have the same structure (Database terminology: are union-compatible. Programming language terminology: have the same type) we can perform set operations.
- All persons who are students or faculty

STUDENT

sid	name
1	Darby
2	Matt
3	Dan
4	Maury

FACULTY

fid	name
1	Darby
12	Youssef
18	Choi

STUDENT ∪ FACULTY

id	name
1	Darby
2	Matt
3	Dan
4	Marty
12	Youssef
18	Choi

Union Example

Emp

eno	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

WorksOn

eno	pno	resp	dur
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P4	Engineer	48
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36
E7	P5	Engineer	23

$$\Pi_{eno}(\text{Emp}) \cup \Pi_{eno}(\text{WorksOn})$$

eno
E1
E2
E3
E4
E5
E6
E7
E8

Set Difference

- ***Set difference*** is a binary operation that takes two relations *R* and *S* as input and produces an output relation that contains all the tuples of *R* that are not in *S*.

- General form:
- $$R - S = \{t \mid t \in R \text{ and } t \notin S\}$$
- where *R* and *S* are relations, *t* is a tuple variable.

- Note that:
  - $R - S \neq S - R$
  - R and S must be union compatible.

Difference –

- Another set operator. Example:

STUDENT

sid	name
1	Sarah
2	Matt
3	Dan
4	Maury

FACULTY

sid	name
1	Sarah
12	Youssef
18	Choi

STUDENT – FACULTY

sid	name
2	Matt
3	Dan
4	Maury

Set Difference Example

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Emp Relation

eno	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

WorksOn Relation

eno	pno	resp	dur
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P4	Engineer	48
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36
E7	P5	Engineer	23

$$\Pi_{eno}(\text{Emp}) - \Pi_{eno}(\text{WorksOn})$$

eno
E4
E8

Question 1: What is the meaning of this query?

Practice Question Cont..

Find the output for the given Relational Algebraic expression

$$\Pi_{T,U}(\sigma_{(V\neq 2)V(V\neq 3)}(S2))$$

S2

T	U	V
1	2	3
2	4	2
1	2	4
3	6	2

Practice Question Cont..

Find the output for the given Relational Algebraic expression

$$\Pi_{P,Q}(\sigma_{(R=3)V(R=5)}(S1))$$

S1

P	Q	R
1	2	3
5	10	4
2	4	3
3	6	5
1	2	5

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Relational Calculus

A **Relational calculus expression** creates a new relation, which is specified in terms of variables that range over rows of the stored database relations

(in tuple calculus) or over columns of the stored relations (in domain calculus).

In a calculus expression,
there is no order of operations to specify how to retrieve the query result.

– A **calculus expression** specifies only **what information the result should contain**.

Relational Calculus

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Relational calculus is considered to be a **nonprocedural language**.

This differs from relational algebra, where we must write a sequence of operations to specify a retrieval request; hence relational algebra can be considered as a procedural way of stating a query.

Tuple Relational Calculus

Tuple Relational Calculus

The tuple relational calculus is based on **specifying a number of tuple variables**.

Each tuple variable usually ranges over a particular database relation, meaning that the variable may take as its value any individual tuple from that relation.

A simple tuple relational calculus query is of the form **{t | COND(t)}** where t is a tuple variable and COND (t) is a conditional expression involving t. **The result of such a query is the set of all tuples t that satisfy COND (t)**

Example 01

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

To find the **first and last names of all employees whose salary is above $50,000,** we can write the following tuple calculus expression:

{t.FNAME, t.LNAME | EMPLOYEE(t) AND t.SALARY>50000}

Example 02 [Good question]

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Relations:

1. **Employee (emp_id, first_name, last_name, dept_id, salary)**
2. **Department (dept_id, dept_name, location)**

Question:

"Given two relations, 'Employee(emp_id, first_name, last_name, dept_id, salary)' and 'Department(dept_id, dept_name, location)', write a **RA and TRC expression** to

find the first and last names of all employees who work in the

department named 'Sales'."

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Example 02 [Good question ..]

Solution:

$$\{t.first\_name, t.last\_name \mid \exists t \in Employee\ (\exists d \in Department\ (t.dept\_id = d.dept\_id \wedge d.dept\_name = 'Sales' \wedge d.emp\_id = t.emp\_id))\}$$

Quantifiers in Tuple Relational Calculus

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Existential Quantifier (∃)
Meaning: "There exists at least one" tuple that satisfies a condition.

Example:
{ e | Employee(e) ∧ ∃p(Project(p) ∧ p.ManagerID = e.EmpID) }
"Find employees who manage at least one project."

Universal Quantifier (∀)
Meaning: "For all" tuples, the condition must hold.

Example:
{ d | Department(d) ∧ ∀e(Employee(e) ∧ e.Dept = d.DeptID ⇒ e.Salary > 50000) }
"Find departments where all employees earn more than $50,000."

Explanation:

The condition EMPLOYEE(t) specifies that the range relation of tuple variable t is EMPLOYEE.

The first and last name (PROJECTION $\pi_{FNAME, LNAME}$) of each EMPLOYEE tuple t that satisfies the condition t.SALARY>50000 (SELECTION $\sigma_{SALARY>50000}$) will be retrieved.

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Practice Questions Click here to open the document